Lecture Notes - Day 10 (Evening)

TypeScript Generics and Scripts, then React: SPA, Components and Vite Setup

Instructor: Abhishek Purohit | Date: 30 Oct 2026 | Prepared for: Sumeet Sharma

What this lecture covers (in teaching order)

1. Setting up the TypeScript demo project again (**npm i**, **src/index.ts**)
2. **Generics** in TypeScript: the duplicated response types problem, then **IEntityResponse<T>**
3. Running TypeScript: **npx tsx**, **watch mode**, and **npm scripts** (**dev**, **build**, **start**); compiling with **tsc** into **dist**
4. **React.js**: library vs framework, and why a library is needed for front-end development
5. The **multi-page** website approach and its drawbacks (no reusability, maintainability, full reloads)
6. **Single Page Applications (SPA)**: behaviour, real examples (Notion, YouTube, Gmail), components instead of pages
7. What a **component** is, and who created React
8. Creating a React app with **Vite** (**npx create-vite@latest**), folder structure, **npm run dev**
9. The **App** component, **main.jsx** as the starting point, and the root **index.html**
10. **npx vs npm**
11. Editing App.jsx (Hello World) and removing the default CSS
12. **Class components vs function components**; creating a class component (**extends Component**)
13. Q&A, career discussion and announcements (non-technical)

How to read these notes

- Normal text is taken from the lecture, converted from spoken words into clear written English.
- Boxes labelled **[Extra - not from the lecture]** are my additions. Treat them as supplementary.
- Grey boxes show **code**; green boxes show **expected output**; red boxes show **common mistakes**.
- The transcript is auto-generated and partly garbled (mixed Hindi/English); I used the meaning that the context makes certain.

Note: No code files were attached for this lecture. All code below is **reconstructed** from what the instructor described and typed (he built it live). Where a value was not stated (for example the product name and price), I used an illustrative value and say so.

1. TypeScript Demo Project: Quick Setup

The instructor copied the morning's **ts-demo** (TypeScript demo) project into a new folder for the evening session and opened it in VS Code. The copy had no **node_modules** folder, so the first step is to create it by installing the dependencies listed in package.json:

```
npm i
```

npm i (short for npm install) reads package.json and creates the **node_modules** folder with all required dependencies. The source code lives in **src/index.ts**. That file already explored (from the morning): creating **variables with data types**, giving **types to function arguments and return values**, and creating **our own data types** with an **interface** or with a **type alias**. Either of those two ways can be used, depending on your requirement.

2. Generics in TypeScript

2.1 The requirement

Suppose you are building a back-end application and, for different APIs, you must decide the **shape of the response data**. The application has several entities: product, customer, order and more. Each entity has different data (a product has different properties from a customer), but the **shape of the response** is similar for all of them:

Key	Meaning (instructor's example)
`status`	The status code: 200, 400, 500 ...
`isSuccess`	A flag to denote response success or failure (status 200 means isSuccess is true)
`timestamp`	When the response was made. **new Date()** without arguments gives the current date and time
`data`	The actual entity data. **Different for each entity**

The instructor first wrote the two objects, thinking only about the data and not about data types yet.

```
const productResponse = {
    status: 200,
    isSuccess: true,
    timestamp: new Date(),
    data: { id: 1, name: "Keyboard", price: 1500, quantity: 85 }   // values illustrative
};

const customerResponse = {
    status: 200,
    isSuccess: true,
    timestamp: new Date(),
    data: { id: 1, name: "Aman", phone: "9090909090", email: "aman@example.com" }
};
```

[Extra - not from the lecture] The product values (Keyboard, 1500) and the customer phone / email are illustrative. The lecture only fixed the keys, the name **Aman**, and later changed the product quantity to 86 to show watch mode.

Phase 1 - Initial approach: one interface per entity response

To give these objects types, the natural solution is a custom data type for each response. The instructor created an **interface** for the product response and one for the customer response, plus an interface for the entity data inside each (a good convention is the prefix **I**).

```
interface IProduct {
    id: number;
    name: string;
    price: number;
    quantity: number;
}

interface IProductResponse {
```

```
    status: number;
    isSuccess: boolean;
    timestamp: Date;
    data: IProduct;          // the entity data has its own type
}

interface ICustomer {
    id: number;
    name: string;
    phone: string;
    email: string;
}

interface ICustomerResponse {
    status: number;
    isSuccess: boolean;
    timestamp: Date;
    data: ICustomer;
}

const productResponse: IProductResponse = { /* as above */ };
const customerResponse: ICustomerResponse = { /* as above */ };
```

- `IProduct` represents the **product data**; `IProductResponse` represents the **entire product response** (status, isSuccess, timestamp and data of type IProduct).
- Annotating the object with `: IProductResponse` makes TypeScript check that it has exactly this shape.
- While typing, the instructor hit an error because he had already declared a type named IProduct earlier; he commented the old code out. (Two declarations with the same name conflict.)

Phase 2 - The problem: code duplication

All the responses have exactly the same structure: status, isSuccess, timestamp, data. Only the type of **data** differs per entity (IProduct, ICustomer, and so on). Yet we are creating a different response type for every entity. We are duplicating the code. We should not do that.

- With 50 entities you would declare 50 almost identical response types.
- If the common shape changes (for example add a **message** key), you must edit every one of them.
- We need to define the operation / shape **once** and let only the **data type** vary. This concept is called **generics**.

2.2 What are generics? The 'add' analogy

The instructor explained the idea with a C++ example. To add integer values, float values and double values, the direct approach is three different functions with different names (integer sum, float sum, double sum): the **operation is the same** and only the **data type changes**. That is code repetition.

```
// Direct approach: same logic repeated for each data type
int    sumInt(int a, int b)          { return a + b; }
float  sumFloat(float a, float b)    { return a + b; }
double sumDouble(double a, double b) { return a + b; }
```

With **generics** you define the logic **once** and pass the working **data type as an argument** when you use it. You specify for which data type the generic function will work (int, float, double). The operation stays the same; only the data type is updated.

[Extra - not from the lecture] The C++ form of this is a template. Illustration only; the lecture described it in words:

```
// Generic approach (C++ template): logic once, data type as a parameter
template <typename T>
T sum(T a, T b) { return a + b; }

sum<int>(2, 3);          // T = int
sum<double>(2.5, 3.5);   // T = double
```

Generics: you generalise the operation and pass the data type as an argument.

Phase 3 - Improved approach: IEntityResponse<T>

The same requirement exists in TypeScript. The response shape is exactly the same for every entity, but the type of **data** changes (product, customer, user, cart, login ...). So the instructor created **one** generic response type, named **IEntityResponse** (not 'product response' or 'customer response', because it is generic for every entity in the application).

```
interface IEntityResponse<T> {
    status: number;        // always a number
    isSuccess: boolean;    // always a boolean
    timestamp: Date;       // always a Date
    data: T;               // type decided by whoever uses this interface
}
```

Explanation

Piece	Meaning
`<T>`	Angular brackets after the name declare a **generic type parameter**. **T** is a **placeholder** for a data type to be supplied later (any name works; T is the common one). It is called a **type parameter**.
`data: T`	The data key does not have a fixed type like IProduct or ICustomer. Its type is whatever is passed for T.
`status, isSuccess, timestamp`	Not generic: fixed types because they are the same for every entity.

Now, when **using** the interface, we must make it specific: pass the actual type in angular brackets (the **type argument**), and it replaces the placeholder T.

```
const productResponse: IEntityResponse<IProduct> = {
    status: 200,
    isSuccess: true,
    timestamp: new Date(),
    data: { id: 1, name: "Keyboard", price: 1500, quantity: 85 }
};

const customerResponse: IEntityResponse<ICustomer> = {
    status: 200,
    isSuccess: true,
    timestamp: new Date(),
    data: { id: 1, name: "Aman", phone: "9090909090", email: "aman@example.com" }
};

console.log(productResponse);
console.log(customerResponse);
```

How it works, step by step

1. **IEntityResponse<IProduct>**: T is replaced by IProduct, so for this object **data** must be of type IProduct.
2. **IEntityResponse<ICustomer>**: T is replaced by ICustomer, so **data** must follow ICustomer.
3. The type is fixed at the point of use. The generic type is flexible, but each object has one definite type (this one is IProduct, that one is ICustomer).
4. No separate IProductResponse or ICustomerResponse is needed any more. A new entity (order, cart, admin ...) only needs its own small data interface, then **IEntityResponse<IOrder>**.

```
{
  status: 200,
  isSuccess: true,
  timestamp: 2026-10-30T12:34:56.789Z,        // current date and time when run
  data: { id: 1, name: 'Keyboard', price: 1500, quantity: 85 }
}
{
  status: 200,
  isSuccess: true,
```

```
  timestamp: 2026-10-30T12:34:56.790Z,
  data: { id: 1, name: 'Aman', phone: '9090909090', email: 'aman@example.com' }
}
```

Why this output appears: TypeScript types exist only at compile time and disappear in the JavaScript that runs, so console.log prints plain objects. The timestamp is whatever moment the code ran.

Type checking in action: when a required key is left out of the data (the instructor showed the **name** missing from the customer data), the editor shows an error, because the object must match ICustomer.

```
const customerResponse: IEntityResponse<ICustomer> = {
    status: 200,
    isSuccess: true,
    timestamp: new Date(),
    data: { id: 1, phone: "9090909090", email: "aman@example.com" }   // 'name' missing
};
// error: Property 'name' is missing in type '{ id: number; phone: string; ... }'
//        but required in type 'ICustomer'.
```

When to use generics: when you have multiple entities (or cases) whose shape / operation is exactly the same and only the data types vary. Then create one generic type, taking the varying type as a parameter.

Concept / Question	Definition or explanation	Purpose / key difference
Generics	Defining logic or a type once, with the data type passed as a parameter	Reuse without duplication
Type parameter (placeholder)	The **T** in IEntityResponse<T>	Stands for a type to be supplied later
Type argument	The actual type in IEntityResponse<IProduct>	Replaces the placeholder at the point of use
Interface (non-generic)	A fixed shape	One shape per interface; leads to duplication when shapes repeat

[Extra - not from the lecture] Generics also work on **functions**. A small illustration (not shown in the lecture):

```
function wrap<T>(data: T): IEntityResponse<T> {
    return { status: 200, isSuccess: true, timestamp: new Date(), data };
}

const r1 = wrap<IProduct>({ id: 1, name: "Keyboard", price: 1500, quantity: 85 });
const r2 = wrap({ id: 2, name: "Aman", phone: "1", email: "a@b.c" });   // T inferred
```

Common mistake: Using **IEntityResponse** without a type argument gives an error: the generic type must be made specific (customer, admin, employee ... whichever it is) before you use it to declare an object.

3. Running TypeScript: tsx, Watch Mode and npm Scripts

3.1 Two ways to run TypeScript

Node runs JavaScript, so TypeScript must become JavaScript. There are two ways:

Way	Tool	Used when
Compile first, then run the JavaScript	`tsc` builds into the **dist** folder; then Node runs it	**Deployment**: live servers, production, staging, testing environments
Run the TypeScript file directly	`tsx`	**Development machine** (local). No manual compile step

3.2 Phase 1 - Running directly with tsx

```
tsx src/index.ts
```

This first failed with an error, for the same reason seen in the morning: by default the system looks for the **tsx** (and **tsc**) command in the **global** installation, not in the project. To force the system to look in the project's **node_modules**, prefix the command with **npx**:

```
npx tsx src/index.ts
```

```
(the two response objects are printed, as shown in section 2)
```

Phase 2 - Problem: restarting manually after each change

When you run a normal program, any change in the source code means you must stop it and run it again. This is especially annoying for a server: each time you change code, you manually stop the server (Ctrl+C) and start it again.

Phase 3 - Improved approach: tsx watch mode

```
npx tsx watch src/index.ts
```

Watch mode keeps the program active and continuously watches the file. Whenever you **save** a change, it automatically restarts the application (a kind of **hot reloading**). The instructor demonstrated by changing a product quantity to 86 and saving: the program restarted and printed the new value.

	Normal mode	Watch mode
Command	`npx tsx src/index.ts`	`npx tsx watch src/index.ts`
After editing the file	Run it again manually	Restarts automatically on save
Use	One-off run	Day-to-day development

Phase 4 - Further improvement: npm scripts

Typing **npx tsx watch src/index.ts** every time is long, and later there will be more commands: build generation, Sonar, unit testing, Docker. So we give a command a **script name** in package.json. This is the **scripts** section (it already had a **test** entry). Developers then run the short name instead of the long command.

```
"scripts": {
    "dev": "tsx watch src/index.ts"
}
```

- Inside package.json scripts you do **not** need **npx**: scripts automatically look into the project's node_modules.
- The name is your choice (**dev**, 'run on local', 'execute' ...). **dev** is the good-practice name for the script that runs the app on a developer's local machine.
- Run a custom script with **npm run <script-name>**.

```
npm run dev
```

It does the same job as the long command: source code changes automatically restart the app.

The build script and the dist folder

To deploy, compile the entire TypeScript application (with subfolders such as controllers and middlewares) into JavaScript using the compiler **tsc**. Run it through npx (the global tsc is not used):

```
npx tsc
```

tsc reads **tsconfig.json**. The **include** and **exclude** settings decide which files are converted; here everything under **src**. The generated JavaScript is saved in the **dist** folder (as set in the config). Inside dist you get files like **index.js**: the same code but **without the data types**. You never edit dist; it exists only for deployment.

Since you may need more options with tsc later, and a **DevOps engineer** who builds the application should not need to know about the compiler, create a **build** script. This gives a **level of abstraction**: the DevOps engineer simply runs the build script.

```
"build": "tsc"
```

The instructor deleted the dist folder and ran **npm run build**; dist was created again with the JavaScript files.

The start script

In production the JavaScript file in dist is what starts the application, with Node:

```
"start": "node dist/index.js"
```

Run it with **npm start**.

Final scripts section

```
"scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "dev": "tsx watch src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js"
}
```

Script	Command to run	Purpose
`dev`	`npm run dev`	Run on the local development machine with automatic restart
`build`	`npm run build`	Compile TypeScript into JavaScript in dist (for deployment)
`start`	`npm start`	Launch the compiled app with Node (production)

start is a **predefined** script key, so it can be run as plain **npm start**. For custom names such as dev and build you must write **npm run** and then the script name.

[Extra - not from the lecture] test is also predefined (`npm test`). The **test** line shown above is the default npm creates; it was already in the file.

Student question - does TypeScript run in production? Answer: ultimately **Node runs only JavaScript**. TypeScript is for development; for deployment (staging, live server, production) the TypeScript is converted to JavaScript and the JavaScript is run with **node dist/index.js**.

The instructor's advice: practise TypeScript, and when you build Express applications you should also implement them in TypeScript, with these configurations.

Tool / command	What it does	When
`tsx`	Runs a .ts file directly (watch mode available)	Development
`tsc`	Compiles TypeScript to JavaScript using tsconfig.json	Build for deployment
`node dist/index.js`	Runs the compiled JavaScript	Production start
`npx`	Runs a command from project node_modules (or a package without installing)	When the command is not global

Common mistake: Running **tsx** or **tsc** directly can fail ('not recognised') when they are not installed globally. Use **npx**, or put the command in a package.json script.

4. Introduction to React.js

4.1 What is React?

The instructor asked: is React a library or a framework? The class answered both ways; the correct answer is **library**. **Angular is a framework** (a vast framework: for the front end you hardly need anything outside Angular), whereas **React is a JavaScript library** used to develop the **front end** of a web application, and specifically to build **Single Page Applications (SPA)**.

Concept / Question	Definition or explanation	Purpose / key difference
React.js	A JavaScript **library** for building the front end (UI) of web applications	Simplifies front-end development; builds SPAs from components
Angular	A front-end **framework**	Complete solution; little need for outside packages
Library vs framework	React covers the UI side; other aspects (for example sending requests and getting responses) need **third-party packages**	With React you look to third-party modules for those jobs

4.2 Why do we need a library?

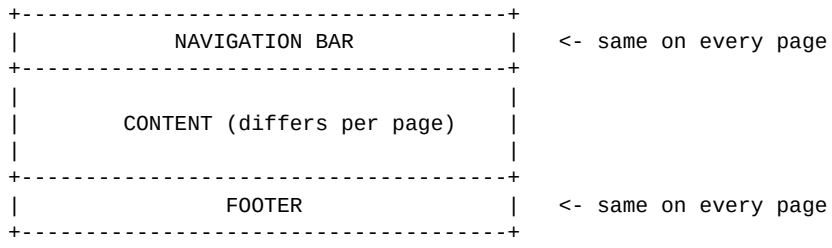
Until now the course focused on the **back end**: APIs and the database. Now the front end begins. You can build a front end with plain **HTML, CSS and JavaScript**: a login page, a registration form, a dashboard, with behaviour added by JavaScript and API calls with **fetch**. The problem is doing it **from scratch**: you build everything from zero. For a real application you then suffer in **scalability** and **development time**, and you need a large amount of effort.

To simplify front-end development you want features, predefined functions and architecture support out of the box. So we use a library, and in this course it is **React**.

5. The Older Multi-Page Approach and Its Drawbacks

Forget React for a moment. Suppose you create a big website for a school with core HTML, CSS and JavaScript. The older approach is to break it into **multiple pages**: home.html, about.html, admissions.html, services, gallery, contact us, and so on.

Requirement: every page has the **same layout**: a **navigation bar** at the top, a **footer** at the bottom, and the page-specific **content** in between.



Drawback 1 - No HTML code reusability

Say the navbar needs 30 lines of HTML and the footer 50 lines. The same 30 + 50 lines must be copied into home.html, about.html, admissions.html and every other page. CSS and JS can be shared, but **HTML cannot be reused** in the older approach. You repeat yourself again and again.

Drawback 2 - Hard to maintain and scale

If the customer asks for a change in the navbar or footer, you must make that change in **each and every HTML file** (50 files means 50 edits). So **maintainability** and **scalability** suffer. Components or UI elements have to be repeated every time.

Drawback 3 - Browser reload on every link click

The instructor opened a real school website. Home, about us, academics, book list, contact us: the layout (top navbar, side menu, footer) is exactly the same, only the content differs. But every time a link is clicked, the **browser reloads**, and it **repaints the entire page**: navbar, sidebar, footer and content, again. Only the content changed, so recreating everything is **not efficient**. If a page takes 5 to 10 seconds to load, every click costs another 5 to 10 seconds: a **performance bottleneck**.

> When the user visits any page, the browser reloads, and due to the reload it repaints the entire page every time. This is not an efficient approach: only the content should be updated.

Fixing this with plain HTML, CSS and JavaScript is possible but needs a lot of effort at the **DOM manipulation** level (you would have to inject and replace elements yourself). The instructor said: these drawbacks become the **features of React**.

Drawback of multi-page approach	Consequence
No HTML reusability	Same navbar / footer code repeated in every file
Not easy to maintain or scale	A small design change means editing every page
Browser reload and repaint on each click	Wasted time, slow, performance bottleneck
Doing better by hand needs heavy DOM manipulation	Large effort and complexity

6. Single Page Application (SPA)

> A single page application has **only a single HTML file**. The entire application is displayed on that one file. When the user navigates between links, the browser is **not reloaded**; based on the URL, new content is displayed **dynamically** on the same page.

6.1 Real examples shown by the instructor

- **Notion:** clicking New chat, Library, a topic: the URL changes (for example /ai, /library) and the content area changes, but the browser does not reload. The sidebar and layout stay the same. Only the **content area is repainted**; the heavy layout was loaded only the first time.
- **YouTube:** click a recommended video: the URL changes (watch?v=video-id) and the content changes without a reload. Searching 'JavaScript tutorial' and opening results also does not reload the browser.
- **Gmail**, Outlook, Drive, Facebook, Instagram, WhatsApp web: switching between Inbox, Sent, Spam changes the content without a page reload. The majority of web applications are SPAs.

Behind the scenes there is a **single HTML file** (index.html). Depending on the URL, different content is rendered into it. It is the same single page: the navbar and footer stay intact, only the content between changes.

```
Multi-page site                          Single Page Application
---------------                          -----------------------
click link                               click link
  -> browser reloads                       -> NO reload
  -> new HTML file downloaded              -> same index.html stays
  -> navbar, sidebar, footer, content      -> only the changing content
     all repainted                            is rendered (URL changes too)
```

6.2 Benefits

- Users get a **desktop / mobile-app-like experience**: links are followed without any reload.
- Best performance: only the necessary content is updated.
- It is nowadays **recommended** that a web application front end be a single page application.

6.3 Components instead of pages

If we build the school website with React, we do **not** create multiple HTML pages. Instead we break the entire front end into different **components**: Home becomes a component, About becomes a component, Admissions a component, Navbar a component and Footer a component. Based on the URL, the matching component is rendered.

Multi-page approach	SPA (React) approach
home.html, about.html, admissions.html ...	Home, About, Admissions **components** inside one HTML file
Navbar code copied into each page	One Navbar component reused
Link click reloads the browser	Link click swaps the component; no reload
Files end in .html	Files end in .js / .jsx

The way of designing an SPA works the same in **Angular** or **Vue.js**: break the whole UI into components and render different components for different URLs. React gives this **out of the box**:

- **Code reusability** out of the box (the HTML reusability problem is solved).
- Easy to **maintain**, and therefore easy to **scale**.
- No browser reload when moving between links.

[Extra - not from the lecture] Switching components according to the URL is called **routing**. In React it is done with an extra package (a router). The lecture only introduced the idea; it will be covered later.

6.4 Who created React?

Facebook developed React. They first built it to create their own single page applications, and later released it publicly. That is why we have the React.js library today.

7. What Is a Component?

> A component is any UI element that is visible on the screen.

- **Atomic level:** a button, a text label, an input field, a heading, a link, an image, a video, a table, audio controls.
- **A whole screen:** a login screen, a dashboard screen, a registration screen is itself a component.
- **A combination of components:** a login screen is made of many components; a dashboard contains a navbar component and a footer component, and those contain link components and image components.

So each and every UI element visible on your screen is a component. In React you work with **components only**: you create components and combine them into screens.

React is specifically a library for this UI work. For other needs, such as sending requests and getting responses, you use **third-party packages**. (Angular, being a vast framework, needs nothing outside of it for the front end.)

8. Creating a React Application with Vite

8.1 Why a tool?

A React application is a **Node application**: it needs a **package.json**, a **src** folder with source code, a **node_modules** folder, and dependencies listed in package.json. You could create the folder, the package.json and install dependencies manually, and set up the folder and file structure yourself, but you should not do that. Use a tool that generates a **boilerplate / starter template**. The tool is **Vite** (pronounced 'veet'), a CLI tool.

Vite can create React, Angular, Vue or plain JavaScript projects. We use it for React.

8.2 The command

```
npx create-vite@latest
```

(The instructor spoke it as 'npx create vite at the rate latest'.) It asks for confirmation to proceed (answer yes), then asks questions:

Prompt	Choice in the lecture	Note
Project name	`my-react-app`	Becomes the project folder name (the instructor used a name like 'my React app')
Select a framework	**React**	Other options include Vue, Angular and vanilla
Select a variant	**JavaScript**	TypeScript variant will be used later, after the basics
Which linter to use	**ESLint**	Linters will be discussed later
Install with npm and start now?	**No**	He installed and started manually

The folder **my-react-app** was created with files and folders generated by Vite. He then opened this folder in VS Code (File > Open Folder).

8.3 Folder structure

```
my-react-app/
|-- package.json       <- scripts and dependencies (it is a Node application)
|-- vite.config.js     <- Vite configuration (customise the tool's behaviour)
|-- index.html         <- the ONLY HTML file of the SPA
|-- README.md          <- description of the application
|-- public/            <- static files such as images served as they are
`-- src/               <- your actual source code and components go here
    |-- main.jsx       <- starting point
    `-- App.jsx        <- the App component
```

- **index.html**: the single HTML file in which the entire front end is displayed. We do not change anything inside it; we only create and deal with components.
- **src**: the real source code. You create your own components inside it.
- **public**: images and similar files to serve.
- **vite.config.js**: configuration for further customising the tool (ESLint was said to come later).

[Extra - not from the lecture] Typical default names (reconstructed; the transcript is unclear on exact file names): the entry point is **main.jsx**, the first component is **App.jsx**, and the config is **vite.config.js**, matching what the instructor described. Vite also adds an **eslint config** file and a **.gitignore**.

8.4 Installing dependencies and running

package.json already has a **scripts** section with **dev**, **build**, **lint** and **preview**, plus **dependencies** (React) and **devDependencies** (the development tools). Install everything, then run the **dev** script:

```
npm i             # creates node_modules (and package-lock.json)
npm run dev       # starts the development server

  VITE ready in ... ms
  Local:   http://localhost:5173/
```

Open the shown URL (local host, port **5173**) in the browser. The instructor saw Vite's default demo screen: a ready-made component that Vite created. Later it will be removed and replaced with our own.

> The life cycle is the same as a Node application: create, install dependencies, run. React runs on top of Node.js.

Common mistake: Run **npm run dev** from **inside the React project folder**. In the wrong folder it will not find package.json. (A student asked and the instructor repeated the steps: open the project, install dependencies with npm i, run npm run dev, open the URL.)

9. The App Component and the Starting Point

In React you create components in **.js** or **.jsx** files (not .html as for pages). The meaning of the **JSX** extension will be explained later.

> Every React application has one already-created component: the **App** component, in **src/App.jsx**. Whenever the application starts, the **App component is rendered (displayed) first**. Even for an application like Facebook, the first component launched is App.

What you saw on the screen after npm run dev was just the App component that Vite generated.

Execution order

1. You run **npm run dev**.
2. Control first goes to **main.jsx** (the starting point).
3. **main.jsx** loads the **App** component.
4. The App component is rendered into the root **index.html** file and appears in the browser.

```
npm run dev --> main.jsx --> App component --> rendered inside index.html --> browser
```

The entire application is displayed at the **root level index.html**; the components are displayed in that one HTML file.

> **[Extra - not from the lecture]** In Vite's default template this wiring looks like this (reconstructed from the template; it was only described in words):

```
// index.html (inside <body>)
//   <div id="root"></div>
//   <script type="module" src="/src/main.jsx"></script>

// src/main.jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

10. npx vs npm

A student asked: we can create a React app using npm too, so why did we use npx? The instructor: **npx is an enhanced version of npm**. Everything npm can do for this purpose, npx can do, but there are things npm cannot do.

Concept / Question	Definition or explanation	Purpose / key difference
npm	A **package manager**: installs and manages dependencies	You must **install** a tool first (for example npm install create-vite), then use it
npx	A **package runner**: executes packages **directly without prior installation**	We never installed the create-vite tool separately; npx ran it directly

- Tools like create-vite are needed only once (to generate the project). With npm you would install them and keep them on disk; with npx you run them directly.
- npx fetches the package dynamically, so it does not fill up your disk storage unnecessarily.

> **[Extra - not from the lecture]** npx also prefers a command from the project's **node_modules**, which is why **npx tsx** worked in section 3 when plain **tsx** did not.

11. Editing the App Component: Hello World

Open **src/App.jsx**. It contains Vite's default HTML-like code. The instructor removed all of it, including the blank tags, wrote his own **div** with an **h1** saying **Hello World**, and removed the default CSS: one import in **main.jsx** (the global CSS) and one in **App.jsx** (the App CSS), because the default styling was being applied. He also removed the unused imports.

```
// src/App.jsx  (after cleaning; reconstructed)
function App() {
  return (
    <div>
      <h1>Hello World</h1>
    </div>
  )
}

export default App
```

- **function App()** is the component; it **returns** the UI to display.
- **export default App** lets main.jsx import it (the same export default / import idea from the Express lectures).
- The HTML-looking code inside return is **JSX** (explained later).

On the browser the page now shows an **h1** with Hello World. Because Vite watches your changes, when you save the source code it **automatically reloads** the browser output (hot reloading), so you do not restart anything.

```
Hello World       <- shown as a heading in the browser
```

Common mistake: If you delete the CSS files but leave their **import** lines in main.jsx or App.jsx, the build shows an error (file not found). Remove the import lines too.

12. Types of Components: Class vs Function

In React JS there are **two types of components**: the **class component** and the **function component**. A component represents a UI element (atomic or an entire screen); the type only depends on **how you write it**.

	Class component	Function component
How created	Using **class syntax**	By simply defining a **function**
Behaviour defined by	The class; object-oriented programming (inheritance, overriding ...)	The function
Weight	Heavier; added complexity from OOP	**Lightweight**, simpler
Performance	Lower	Better (as stated by the instructor)
History	Present **from the beginning** of React; the only way to create components then	Introduced in later updates (they gained the needed features later)
Use today	Only when a project requires it or OOP is needed	**Majority of modern applications**

Example: to create a button, a navbar, a footer or a login screen, a developer decides: write **class syntax** (class component) or **function syntax** (function component). The **App** component in the project is a **function component** (the class answered 'function' when asked).

The instructor said you should know class components, because you may meet them in a project, and the fundamental features of React (for example the **lifecycle**) have been available in classes from the beginning. For understanding, the course will explore class components first and then implement the same things in function components, and from then on move to function components. Majority of people use functions, and we will too.

Creating a class component

Steps taught (the file and class are for a Navbar):

1. Create a **new file** in **src** for the component with a .js or .jsx extension, named after the component: **Navbar.jsx** (or Footer.jsx, Home.jsx). **Each component has its own separate file** (a student asked, and the instructor confirmed).
2. Inside it, create a **class with the same name** as the component: **class Navbar**. By recommendation the file name and class name match.
3. A plain class is **not** automatically a React component. To give it the properties of a React component, it must **inherit** from the **Component** class, which is already defined in the **react** package. So import **Component** and use **extends**.

```
// src/Navbar.jsx
import { Component } from 'react';

class Navbar extends Component {
    // the UI (links ...) and behaviour of the navbar will be defined here
}
```

Without the inheritance (extends Component), the class will not be recognised as a UI component. By inheriting from the Component class, the Navbar class gets all the properties needed to be recognised and treated as a UI component. The same syntax applies to every class component: class Footer extends Component, class Home extends Component, class Login extends Component.

[Extra - not from the lecture] The lecture stopped here: the body of the class was not written. A class component must define a **render()** method that returns the JSX to display, and be exported (`export default Navbar;`) to be used elsewhere. Expect that in the next lecture. Do not expect the empty class above to display anything yet.

Common mistake: Writing only **class Navbar { }** and assuming it is a component is wrong: you must extend the React Component class. File name and class name should also match the component name.

13. Discussion and Announcements (Non-Technical)

Note: The final part of the transcript is a free discussion and then announcements by another speaker. It is very garbled; this is my best reading. Confirm details with official messages.

13.1 Student questions

- Is each component in a separate file? **Yes.**
- A student asked about the difference between **.js and .jsx** files; the instructor indicated JSX would be explained later.

13.2 Career discussion (informal)

- Work-life balance varies by client. UK and European clients were described as relatively relaxed, with limited overlap time; US clients can bring uncertainty and sudden changes.
- With AI tools, expectations on developers are changing: feature estimates drop from a week to a day or two. A developer must increasingly act as an **orchestrator**: control how things work and focus on architecture, and govern the AI, because **AI can make mistakes**.
- Hiring is changing; companies may prefer one person working with AI. Clients increasingly expect **full-stack** skills plus deployment pipelines, testing automation and cloud (AWS / Azure) knowledge. The term **forward deployed engineer** came up.
- Advice: be able to work with any technology; learn continuously.

13.3 Announcements

- Next lecture starts at **8:00 AM** (normal schedule).

- A **picnic**: respond (react) to the message so arrangements can be made. A **Diwali celebration** is also planned (date garbled; around 7 November was mentioned): please confirm attendance.
- Lectures are scheduled until the **6th**; the **7th and 8th** were mentioned as holidays; evaluations around the 9th and 10th (unclear).
- **Project topics:** teams should discuss and coordinate topics so they do not repeat. A form will be shared after the morning lecture, with about **one hour** to submit ideas; topics will then be finalised and shared (maybe by that afternoon or the next day), followed by discussion of functionality and design. Plenty of time is available.
- A **presentation topic** is expected, most probably on **Monday**. The **aptitude lecture** on Monday was not confirmed.

14. Source Coverage Checklist

Checked against the transcript from beginning to end:

Topic in transcript	Covered in section
Copy ts-demo to new folder, npm i, node_modules, src/index.ts, morning recap	1
Response objects (status, isSuccess, timestamp, data) for product and customer	2.1
Separate interfaces IProduct / IProductResponse / ICustomer / ICustomerResponse; duplication problem	2 (Phase 1 - 2)
Generics concept; C++ int / float / double analogy	2.2
IEntityResponse<T>, type parameter, type argument, usage, missing-key error	2 (Phase 3)
tsx, npx tsx, global vs project lookup error	3.2 (Phase 1)
tsx watch mode and hot restart (quantity 86 demo)	3 (Phases 2 - 3)
package.json scripts: dev, npm run, naming, abstraction	3 (Phase 4)
tsc build, dist folder, build script, DevOps abstraction, start script, npm start vs npm run	3 (build and start)
Student question: TypeScript vs JavaScript at runtime	3
React: library vs framework, front-end purpose, why from scratch is hard	4
Multi-page approach, reuse (navbar 30 lines, footer 50 lines), maintainability, reload/repaint	5
SPA definition, Notion / YouTube / Gmail examples, URL-based content, benefits	6.1 - 6.2
Components instead of pages; Angular / Vue same idea; React out-of-box benefits; created by Facebook	6.3 - 6.4
What is a component (atomic to whole screen); third-party packages vs Angular	7
Vite, npx create-vite@latest, prompts, folder structure, npm i, npm run dev, port 5173	8
App component, first rendered, main.jsx starting point, root index.html	9
npx vs npm	10
Hello World edit, removing CSS and unused imports, hot reload	11
Class vs function components, history, performance, App is a function, creating class component with extends Component	12
Student Q&A, career discussion, announcements	13

15. Revision Material

15.1 Summary of major topics

- **Generics:** define a shape or operation once and pass the varying data type as a parameter: **IEntityResponse<T>** with **IEntityResponse<IProduct>**. Avoids duplicate types.
- **Running TypeScript: npx tsx file.ts** for development, **tsx watch** restarts on save, **tsc** compiles into **dist**, Node runs the JavaScript in production.
- **npm scripts: dev** (npm run dev), **build** (npm run build), **start** (npm start); scripts need no npx.
- **React** is a JavaScript **library** (by Facebook) for building SPAs from **components**.
- Multi-page websites suffer from no HTML reuse, hard maintenance and full reloads; **SPAs** use one HTML file and swap content by URL without reload.
- A **component** is any visible UI element, from a button to a whole screen.
- Create a React app with **npx create-vite@latest**, install with **npm i**, run with **npm run dev** (port 5173).
- **main.jsx** starts, renders **App** into **index.html**; edit **App.jsx** and Vite hot-reloads.
- **npm** installs packages; **npx** runs packages without installing.
- Components are **class** (extends Component) or **function**; function components are lightweight and the modern choice.

15.2 Important definitions and syntax

Concept / Question	Definition or explanation	Purpose / key difference
Generic	Type or function that takes a data type as a parameter	Reuse for many data types
`interface X<T> { data: T }`	Generic interface with type parameter T	One shape, many data types
`X<IProduct>`	Using the generic with a type argument	Fixes T for that object
`npx tsx watch src/index.ts`	Run TS directly; restart on save	Development
`npx tsc`	Compile TS to JS in dist	Build for deployment
`npm run dev / build`	Run a custom script	Short names for long commands
`npm start`	Run the predefined start script	Launch the app (production)
SPA	App with a single HTML file; no reload on navigation	Fast, app-like experience
Component	A visible UI element (atomic or whole screen)	Building block of a React app
Vite	CLI tool to generate and run front-end projects	Creates React starter template
`App.jsx / main.jsx`	First component / starting point	main.jsx loads App into index.html
`class X extends Component`	Class component	Needs inheritance to be a component

15.3 Key differences

Compare	Difference
Library vs framework	React is a UI library needing third-party packages; Angular is a full framework.
Multi-page vs SPA	Multi-page: many HTML files, reload on each link, repeated HTML. SPA: one HTML file, no reload, reusable components.
npm vs npx	npm installs / manages packages; npx runs packages directly without installing.
tsx vs tsc	tsx runs TypeScript directly (development); tsc compiles to JavaScript (build).
Normal vs watch mode	Watch mode restarts automatically when files change.
Class vs function component	Class: OOP, heavier, since the beginning. Function: lightweight, better performance, modern default.
Interface per entity vs generic interface	Per entity duplicates the same shape; generic defines it once.
.html pages vs components	Pages are separate HTML files; components are .js / .jsx files rendered into one HTML file.

15.4 Common mistakes to avoid

Mistake	Correction
Creating one response type per entity	Create one generic type with a type parameter
Using a generic type without a type argument	Always pass it: IEntityResponse<IProduct>
Running tsx or tsc directly and getting 'not found'	Use npx, or a package.json script
Using npx inside package.json scripts	Not needed; scripts look in node_modules
Forgetting **npm run** for dev and build	Only start (and test) work without 'run'
Editing files in dist	dist is generated; edit src and rebuild
Writing class Navbar { } only	Extend the Component class from 'react'
Mismatch between file name and class name	Name both after the component (Navbar.jsx, class Navbar)
Running npm run dev outside the project folder	Run it inside the React project (where package.json is)
Leaving CSS import lines after deleting the CSS	Remove the import lines as well
Forgetting npm i after generating the project	Install dependencies to create node_modules

15.5 Interview questions and answers

Questions marked (E) go beyond what the instructor said; the rest follow the lecture.

Question	Answer
What are generics?	Defining a type or operation once and passing the data type as a parameter, so it works with different data types without duplicating code.
Why IEntityResponse<T> instead of separate response interfaces?	The response shape is the same for every entity; only data differs. A generic type avoids duplication.
What is T?	A type parameter: a placeholder for the actual type supplied when the generic is used.
How do you run a TypeScript file in development?	npx tsx src/index.ts, or npx tsx watch for automatic restart.
How is TypeScript deployed?	Compile with tsc into dist and run the JavaScript with node dist/index.js.
Why define npm scripts?	Short names for long commands, and an abstraction so others (for example DevOps) need not know the details.
Which npm scripts can run without 'run'?	Predefined ones like start (and test); custom ones need npm run.
Is React a library or framework?	A JavaScript library for building front-end UI; Angular is the framework.

Question	Answer
What is a single page application?	An application with one HTML file where navigation changes content dynamically without a browser reload.
Drawbacks of multi-page websites?	No HTML reusability, difficult to maintain and scale, full page reload and repaint on each navigation.
Benefits of React for SPAs?	Code reusability through components, easier maintenance and scaling, no reload between links.
What is a component?	Any UI element visible on screen, from a button to an entire screen.
Who created React?	Facebook, first for its own applications, later released publicly.
How do you create a React app?	npx create-vite@latest, choose React and JavaScript, npm i, then npm run dev.
What is the first component rendered?	App; main.jsx loads it into the root index.html.
npm vs npx?	npm installs and manages packages; npx executes packages directly without prior installation.
Class vs function component?	Class uses class syntax and OOP (extends Component); function is a plain function, lightweight, better performance and mostly used now.
(E) What is the Vite dev server doing on save?	It performs hot module reloading: only changed modules update in the browser, without a full reload.
(E) What does 'type erasure' mean for generics?	Type arguments exist only at compile time; the generated JavaScript has no types.

15.6 Practice exercises (solutions not included)

Easy

1. Write an interface **IOrder** with id, item and amount, and declare an object of type **IEntityResponse<IOrder>**.
2. Explain in one line the difference between **npm run dev** and **npm start**.
3. List the three drawbacks of the multi-page approach.

Medium

1. Add **dev**, **build** and **start** scripts to a TypeScript project and run all three.
2. Change a value while **npm run dev** is running and observe the restart.
3. Create a Vite React app, remove the default CSS and imports, and display your name in an h1 in App.jsx.
4. Explain with a diagram how main.jsx, App and index.html are connected.

Challenging

1. Write a generic function **wrap<T>(data: T)** that returns an **IEntityResponse<T>** and call it with two different types.
2. Create **Navbar.jsx** and **Footer.jsx** as class components (extends Component) and describe what is still missing for them to display.
3. Compare how you would add a new 'Contact' page in a multi-page site and in an SPA.
4. Explain why Node still runs JavaScript even though you wrote TypeScript.

15.7 Final revision checklist

[ ] I can explain what generics are and why IEntityResponse<T> removes duplication.

[ ] I can declare objects using a generic interface with a type argument.

[ ] I can run TypeScript with npx tsx, use watch mode, and explain why npx is needed.

[ ] I can add dev, build and start scripts and run them correctly.

- [ ] I know why tsc output goes to dist and that Node runs the JavaScript.
- [ ] I can say why React is a library and Angular a framework.
- [ ] I can list the drawbacks of multi-page websites.
- [ ] I can explain what an SPA is, with examples, and what happens on a link click.
- [ ] I can define a component at atomic and screen level.
- [ ] I can create a React app with Vite and run it on port 5173.
- [ ] I can explain the flow main.jsx, App component, root index.html.
- [ ] I know the difference between npm and npx.
- [ ] I can compare class and function components and write a class component skeleton.